SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	M Dhanush Kumar	**Reg. No.:**	192525318
Date:	28-07-2026	**Duration:**	60 minutes

Code of Conduct

I ____(M Dhanush Kumar/192525318)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M Dhanush Kumar

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modeling	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification	SRS is well-structured, complete, professionally organized, and	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

(SRS) Documentation	includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis	/5
Functional and Non-Functional Requirement Identification	/5
Actor Identification and Use Case Modeling	/5
Software Requirement Specification (SRS) Documentation	/5
Assumptions, Constraints, and Requirement Validation	/5
TOTAL	/5

CO2- Assessment tool -1

Software Requirement Specification (SRS)

Title : Smart Vertical Farming Automation Platform

NAME : M. Dhanush Kumar

REG NO : 192525318

1. Study and Analyze the Scenario

Problem Statement:

Traditional farming requires large land areas, constant monitoring, and manual irrigation. The Smart Vertical Farming Automation Platform automates crop monitoring, irrigation, lighting, and environmental control using IoT.

Business Domain

- Smart Agriculture
- Vertical Farming
- IoT-Based Automation
- Precision Farming

Stakeholders

- Farm Owner
- Farm Manager
- Workers
- Customers
- System Administrator
- Maintenance Team

Existing Challenges

- Manual monitoring
- High water usage
- Uncontrolled environment
- High labor cost
- Low productivity
- Late disease detection

System Objectives

- Automate irrigation and lighting
- Monitor environment
- Improve productivity
- Reduce water and energy use
- Real-time monitoring

2. Functional Requirements

- User Registration/Login
- Dashboard
- Monitor Sensors
- Automatic Irrigation
- Smart Lighting
- Crop Management
- Alerts
- Reports
- User Management
- Device Management

Non-Functional Requirements

- Performance
- Security
- Reliability
- Usability
- Scalability
- Maintainability
- Availability

3. Actors and Use Cases

Primary Actors

- Farm Owner
- Farm Manager
- Worker
- Administrator

Secondary Actors

- IoT Sensors
- Water Pump
- Lighting System
- Notification Service

Major Use Cases

- Login
- Monitor Farm
- View Sensor Data
- Control Irrigation
- Control Lighting
- Manage Crops
- Receive Alerts
- Generate Reports
- Manage Users
- Configure Devices

4. Software Requirement Specification (SRS)

Introduction

Automates vertical farming using IoT for efficient crop production.

Problem Description

Manual farming wastes water and requires continuous monitoring.

Scope of the System

Smart monitoring, irrigation, lighting, crop management, reports, user management.

Functional Requirements

Authentication, monitoring, irrigation, lighting, notifications, reports.

Non-Functional Requirements

Security, performance, reliability, scalability, availability, maintainability.

Use Case Descriptions

Login, Monitor Farm, Irrigation Control, Lighting Control, Generate Reports, Receive Alerts.

Data Requirements

User details, crop information, sensor data, irrigation records, reports.

External Interface Requirements

Mobile app, web dashboard, IoT sensors, pump controller, LED controller.

System Features

Real-time monitoring, smart irrigation, automated lighting, analytics, alerts.

5. Assumptions and Constraints

Assumptions

- Stable Internet
- Installed IoT sensors

- Continuous power
- Trained users
- Valid sensor data

Constraints

- Limited budget
- Internet dependency
- Hardware compatibility
- Sensor accuracy
- Maintenance cost
- Time limit

6. Validate Requirement Completeness and Consistency

- All stakeholder requirements identified
- Functional and non-functional requirements complete
- Actors and use cases defined
- Requirements are clear and consistent